



Solution - Data Engineering

CAPSTONE PROJECT



End-to-End Modern Data Pipeline



STEP 1 – EXTRACT DATA

Goal

Extract product data from the Fake Store API and save it as both JSON and CSV files.

Expected Deliverables

- ✓ products.json
- ✓ products.csv
- ✓ Python extraction script
- ✓ Project folder structure

1. Create the Project Folder

Create the following folders.

```
Capstone_Project/  
├── data/  
│   ├── raw/  
│   └── processed/  
├── scripts/  
├── docs/  
├── sql/  
├── airflow/  
│   └── dags/  
├── dbt_project/  
├── powerbi/  
├── tests/  
└── .github/  
    └── workflows/
```

```
| requirements.txt
| .gitignore
| README.md
```

1. Open the Project in VS Code

```
File
→ Open Folder
→ Capstone_Project
```

3. Create a Virtual Environment

Open a terminal.

Windows

```
python -m venv .venv
```

Activate it

```
.venv\Scripts\activate
```

You should see

```
(.venv)
```

at the beginning of the terminal.

4. Create requirements.txt

```
requests
pandas
```

Install them

```
pip install -r requirements.txt
```

Verify

```
pip list
```

You should see

```
pandas  
requests
```

5. Create the Extraction Script

Create

```
scripts/  
extract.py
```

Paste the following code.

```
import json  
from pathlib import Path  
  
import pandas as pd  
import requests  
  
API_URL = "https://fakestoreapi.com/products"  
  
RAW_DATA_DIR = Path("data/raw")  
RAW_DATA_DIR.mkdir(parents=True, exist_ok=True)  
  
def extract_products():  
    """  
    Extract product data from the Fake Store API.  
    """  
  
    print("Fetching data from Fake Store API...")  
  
    response = requests.get(API_URL, timeout=30)  
    response.raise_for_status()  
  
    products = response.json()  
  
    print(f"Retrieved {len(products)} products.")  
  
    return products  
  
def save_json(products):
```

```

json_path = RAW_DATA_DIR / "products.json"

with open(json_path, "w", encoding="utf-8") as file:
    json.dump(products, file, indent=4)

print(f"JSON saved to {json_path}")

def save_csv(products):
    csv_path = RAW_DATA_DIR / "products.csv"

    df = pd.json_normalize(products)

    df.to_csv(csv_path, index=False)

    print(f"CSV saved to {csv_path}")

def main():
    products = extract_products()

    save_json(products)
    save_csv(products)

    print("Extraction completed successfully.")

if __name__ == "__main__":
    main()

```

This ->

- uses functions
- creates folders automatically
- handles HTTP errors
- uses Path instead of hard-coded paths
- includes clear logging
- is easier to reuse in Airflow later

6. Run the Script

```
python scripts/extract.py
```

Expected output

```
Fetching data from Fake Store API...
Retrieved 20 products.
JSON saved to data/raw/products.json
CSV saved to data/raw/products.csv
Extraction completed successfully.
```

7. Verify

Your project should now look like this.

```
Capstone_Project/
├── data/
│   ├── raw/
│   │   ├── products.csv
│   │   └── products.json
│   └── processed/
├── scripts/
│   └── extract.py
└── requirements.txt
```

Open the CSV and verify that columns such as the following are present:

```
id
title
price
description
category
image
rating.rate
rating.count
```

Step 1 Checklist

- ✓ Project folder created
- ✓ Virtual environment created

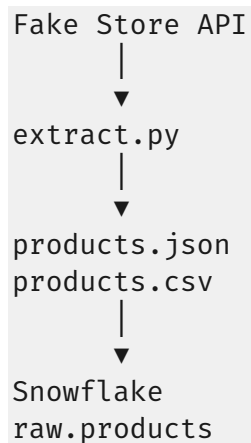
- ✓ Dependencies installed
- ✓ API extraction script completed
- ✓ JSON generated
- ✓ CSV generated
- ✓ Script runs without errors

STEP 2 – LOAD RAW DATA INTO SNOWFLAKE

Goal

Load the extracted products.csv into the raw.products table in Snowflake.

Architecture So Far



Prerequisites

Before writing any code, make sure you have:

- ✓ A Snowflake account
- ✓ A Warehouse
- ✓ A Database
- ✓ A User with the necessary permissions

1. Install the Snowflake Connector

Update your requirements.txt to include:

```
requests  
  
pandas  
snowflake-connector-python  
python-dotenv
```

Then install the dependencies:

```
pip install -r requirements.txt
```

2. Create the Configuration File

Create a **.env** file in the project root:

```
Capstone_Project/  
├── .env
```

Add your Snowflake credentials:

```
SNOWFLAKE_ACCOUNT=your_account  
SNOWFLAKE_USER=your_username  
SNOWFLAKE_PASSWORD=your_password  
SNOWFLAKE_WAREHOUSE=COMPUTE_WH  
SNOWFLAKE_DATABASE=ECOMMERCE_DB  
SNOWFLAKE_SCHEMA=RAW  
SNOWFLAKE_ROLE=ACCOUNTADMIN
```

3. Update .gitignore

Create or update .gitignore:

```
.venv/  
.env  
__pycache__/  
*.pyc  
.airflow/  
dbt_project/target/  
dbt_project/logs/
```


4. Create the Database

Open a Snowflake worksheet and run:

```
CREATE DATABASE IF NOT EXISTS ECOMMERCE_DB;
```

5. Create the Schema

```
USE DATABASE ECOMMERCE_DB;
```

```
CREATE SCHEMA IF NOT EXISTS RAW;
```

6. Create the Products Table

Run:

```
CREATE OR REPLACE TABLE RAW.PRODUCTS (  
  ID INTEGER,  
  TITLE STRING,  
  PRICE FLOAT,  
  DESCRIPTION STRING,  
  CATEGORY STRING,  
  IMAGE STRING,  
  RATING_RATE FLOAT,  
  RATING_COUNT INTEGER  
);
```

Notice that we split the nested `rating` object into two columns (`RATING_RATE` and `RATING_COUNT`) because your CSV contains `rating.rate` and `rating.count`.

7. Verify the Table

Run:

```
SHOW TABLES IN SCHEMA RAW;
```

You should see:

```
PRODUCTS
```

Step 2 Checklist

By the end of this step, you should have:

- ✓ Snowflake connector installed
- ✓ .env file created
- ✓ .gitignore updated
- ✓ ECOMMERCE_DB database created
- ✓ RAW schema created
- ✓ RAW.PRODUCTS table created

STEP 3 – LOAD THE CSV INTO SNOWFLAKE

Goal

Create a Python script that:

- Reads products.csv
- Connects to Snowflake
- Clears the existing data (optional for now)
- Loads all products into RAW.PRODUCTS

Project Structure

Your project should now look like this:

```
Capstone_Project/
├── data/
│   └── raw/
│       └── products.csv
├── scripts/
│   ├── extract.py
│   └── load_snowflake.py
├── .env
└── requirements.txt
```

└─ .gitignore

Step 1. Create load_snowflake.py

Create:

```
scripts/  
  load_snowflake.py
```

Paste the following code.

```
import os

import pandas as pd
import snowflake.connector
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Read CSV
df = pd.read_csv("data/raw/products.csv")

# Connect to Snowflake
conn = snowflake.connector.connect(
    account=os.getenv("SNOWFLAKE_ACCOUNT"),
    user=os.getenv("SNOWFLAKE_USER"),
    password=os.getenv("SNOWFLAKE_PASSWORD"),
    warehouse=os.getenv("SNOWFLAKE_WAREHOUSE"),
    database=os.getenv("SNOWFLAKE_DATABASE"),
    schema=os.getenv("SNOWFLAKE_SCHEMA"),
    role=os.getenv("SNOWFLAKE_ROLE"),
)

cursor = conn.cursor()

try:
    print("Connected to Snowflake.")

    # Optional: clear table before loading
    cursor.execute("TRUNCATE TABLE RAW.PRODUCTS")

    insert_sql = """
    INSERT INTO RAW.PRODUCTS
    (
```

```

        ID,
        TITLE,
        PRICE,
        DESCRIPTION,
        CATEGORY,
        IMAGE,
        RATING_RATE,
        RATING_COUNT
    )
VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
"""

for _, row in df.iterrows():
    cursor.execute(
        insert_sql,
        (
            int(row["id"]),
            row["title"],
            float(row["price"]),
            row["description"],
            row["category"],
            row["image"],
            float(row["rating.rate"]),
            int(row["rating.count"]),
        ),
    )

conn.commit()

print(f"Successfully loaded {len(df)} rows.")

finally:
    cursor.close()
    conn.close()
    print("Connection closed.")

```

Step 2. Verify Your .env

Example:

```

SNOWFLAKE_ACCOUNT=xxxxxxxx.us-east-1
SNOWFLAKE_USER=YOUR_USERNAME
SNOWFLAKE_PASSWORD=YOUR_PASSWORD
SNOWFLAKE_WAREHOUSE=COMPUTE_WH
SNOWFLAKE_DATABASE=ECOMMERCE_DB
SNOWFLAKE_SCHEMA=RAW

```

```
SNOWFLAKE_ROLE=ACCOUNTADMIN
```

Your SNOWFLAKE_ACCOUNT should match the account identifier shown in your Snowflake URL. For example:

```
https://craftsoft.us-east-1.snowflakecomputing.com
```

Then:

```
SNOWFLAKE_ACCOUNT=abc12345.us-east-1
```

Step 3. Run the Loader

```
python scripts/load_snowflake.py
```

Expected output:

```
Connected to Snowflake.  
Successfully loaded 20 rows.  
Connection closed.
```

Step 4. Verify in Snowflake

Run:

```
SELECT COUNT(*)  
FROM RAW.PRODUCTS;
```

Expected:

COUNT

20

Then inspect the data:

```
SELECT *  
FROM RAW.PRODUCTS  
LIMIT 10;
```

You should see columns like:

	ID TITLE	PRICE	CATEGORY	RATING_RATE	RATING_COUNT
1	Fjallraven Backpack	109.95	men's clothing	3.9	120

Step 5. Commit to Git

```
git add .  
git commit -m "Load raw product data into Snowflake"
```

Step 3 Checklist

- ✓ Created load_snowflake.py
- ✓ Connected to Snowflake using environment variables
- ✓ Loaded products.csv into RAW.PRODUCTS
- ✓ Verified all rows in Snowflake
- ✓ Committed the changes to Git

STEP 4 – CREATE AND CONFIGURE THE DBT PROJECT

Goal

Create a dbt project that connects to Snowflake and transforms the raw data into clean, tested models.

What we'll accomplish in this step

- ✓ Install dbt
- ✓ Create the dbt project
- ✓ Configure the Snowflake connection
- ✓ Test the connection

- ✓ Create a clean folder structure

We won't build any models yet. We'll do that in the next step after verifying the setup.

Project Structure

Your project will now look like this:

```
Capstone_Project/  
├── data/  
├── scripts/  
├── airflow/  
├── docs/  
├── powerbi/  
├── sql/  
├── tests/  
├── dbt_project/  
│   ├── dbt_project.yml  
│   ├── profiles.yml  
│   ├── packages.yml  
│   ├── models/  
│   │   ├── staging/  
│   │   ├── intermediate/  
│   │   └── marts/  
│   ├── tests/  
│   ├── macros/  
│   ├── snapshots/  
│   ├── seeds/  
│   └── analyses/  
├── requirements.txt  
└── .env
```

Step 1. Install dbt

Update **requirements.txt**

```
requests  
pandas  
snowflake-connector-python
```

```
python-dotenv  
dbt-core  
dbt-snowflake
```

Install everything.

```
pip install -r requirements.txt
```

Verify:

```
dbt --version
```

Example output:

```
Core:  
- installed: 1.10.x  
  
Plugins:  
- snowflake: 1.10.x
```

Step 2. Create the dbt Project

From the **Capstone_Project** folder:

```
dbt init dbt_project
```

Choose

```
1
```

for Snowflake.

After initialization, your project becomes:

```
dbt_project/  
|  
|— analyses/  
|— macros/  
|— models/  
|— seeds/  
|— snapshots/
```



```
|— tests/  
|— dbt_project.yml  
|— packages.yml
```

Step 3. Create profiles.yml

Inside

```
dbt_project/
```

create

```
profiles.yml
```

Paste:

```
dbt_project:  
  target: dev  
  
  outputs:  
  
    dev:  
      type: snowflake  
  
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"  
      user: "{{ env_var('SNOWFLAKE_USER') }}"  
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"  
  
      role: "{{ env_var('SNOWFLAKE_ROLE') }}"  
  
      database: "{{ env_var('SNOWFLAKE_DATABASE') }}"  
      warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"  
      schema: ANALYTICS  
  
      threads: 4  
  
      client_session_keep_alive: False
```

Notice that the target schema is **ANALYTICS**. We'll keep the raw tables in RAW and have dbt build transformed models in a separate schema.

Step 4. Create the Analytics Schema

Run this once in Snowflake:

```
CREATE SCHEMA IF NOT EXISTS ANALYTICS;
```

Step 5. Point dbt to the Local Profile

By default, dbt looks for profiles.yml in your home directory. Since we're keeping it inside the project, run dbt with the profile directory:

Windows

```
dbt debug --profiles-dir .
```

Run this command from inside the dbt_project directory.

If everything is configured correctly, you should see:

```
Connection test: OK connection ok  
All checks passed!
```

Step 6. Configure dbt_project.yml

Replace the generated file with:

```
name: "dbt_project"  
version: "1.0.0"  
  
config-version: 2  
  
profile: "dbt_project"  
  
model-paths: ["models"]  
seed-paths: ["seeds"]  
snapshot-paths: ["snapshots"]  
macro-paths: ["macros"]  
test-paths: ["tests"]  
analysis-paths: ["analyses"]  
  
target-path: "target"  
clean-targets:  
  - "target"  
  - "dbt_packages"
```

```
models:

  dbt_project:

    staging:
      +materialized: view

    intermediate:
      +materialized: view

    marts:
      +materialized: table
```

This configuration ensures:

- **Staging models** are materialized as **views**
- **Intermediate models** are materialized as **views**
- **Gold (marts) models** are materialized as **tables**

This follows common dbt best practices.

Step 7. Organize the Models Folder

Create:

```
models/
├── staging/
├── intermediate/
└── marts/
```

Step 4 Checklist

You should now have:

- ✓ dbt installed
- ✓ dbt project initialized

- ✓ profiles.yml configured
- ✓ ANALYTICS schema created
- ✓ dbt debug --profiles-dir . passes successfully
- ✓ dbt_project.yml configured
- ✓ Model folders (staging, intermediate, marts) created

STEP 5 – BUILD THE STAGING LAYER

Goal

We'll create:

```
models/  
└─ staging/  
    ├── sources.yml  
    ├── schema.yml  
    └─ stg_products.sql
```

After this step you'll be able to run:

```
dbt run  
dbt test
```

successfully.

Step 1. Create sources.yml

Create

```
dbt_project/  
└─ models/  
    └─ staging/  
        └─ sources.yml
```

Paste the following:

```
version: 2  
  
sources:
```

```

- name: raw
  description: >
    Raw layer containing product data extracted directly from the Fake S
    tore API.
    This data is loaded into Snowflake without any business transformati
    ons and
    serves as the foundation for downstream dbt models.
  database: ECOMMERCE_DB
  schema: RAW

  tables:
    - name: products
      description: >
        Raw products table containing product details, pricing informati
        on,
        category, image URL, and customer rating metrics loaded from the
        API.

```

Step 2. Create stg_products.sql

Create

```

dbt_project/
├── models/
│   ├── staging/
│   │   └── stg_products.sql

```

```

SELECT

    id,
    title,
    description,
    category,
    price,
    image,
    rating_rate,
    rating_count

FROM {{ source('raw', 'products') }}

```

Notice we're simply selecting the columns because the Python loader already flattened the nested rating object.

Step 3. Create schema.yml

This file contains:

- documentation
- column descriptions
- tests

Create

```
dbt_project/
├── models/
│   └── staging/
│       └── schema.yml
```

Paste this:

```
version: 2

models:
  - name: stg_products
    description: >
      Staging model that cleans and standardizes raw product data loaded from
      the Fake Store API. This model preserves one record per product and
      prepares the data for downstream analytical models.
    columns:
      - name: id
        description: Unique identifier for each product.
        tests:
          - unique
          - not_null
      - name: title
        description: Name of the product.
        tests:
          - not_null
      - name: description
        description: Detailed product description.
      - name: category
        description: Product category assigned by the API.
```

```

    tests:
      - not_null

  - name: price
    description: Product selling price in USD.
    tests:
      - not_null

  - name: image
    description: URL of the product image.

  - name: rating_rate
    description: Average customer rating.
    tests:
      - not_null

  - name: rating_count
    description: Total number of customer reviews.
    tests:
      - not_null

```

Step 4. Run dbt

From the dbt_project directory:

```
dbt run --profiles-dir .
```

Expected output:

```

1 of 1 START sql view model ANALYTICS.stg_products
1 of 1 OK created sql view model ANALYTICS.stg_products

```

Step 5. Run Tests

```
dbt test --profiles-dir .
```

Expected:

```

PASS not_null_stg_products_id
PASS unique_stg_products_id
PASS not_null_stg_products_title
PASS not_null_stg_products_category
PASS not_null_stg_products_price

```

```
PASS not_null_stg_products_rating_rate
PASS not_null_stg_products_rating_count
```

Done.

Step 6. Verify in Snowflake

Run:

```
SELECT *
FROM ANALYTICS.STG_PRODUCTS
LIMIT 10;
```

You should see

ID	TITLE	CATEGORY	PRICE	RATING_RATE	RATING_COUNT
1	Fjallraven Backpack	men's clothing	109.95	3.9	120

Folder Structure

Your project should now look like this:

```
dbt_project/
├── models/
├── staging/
│   ├── schema.yml
│   ├── sources.yml
│   └── stg_products.sql
├── intermediate/
└── marts/
```

Step 5 Checklist

- ✓ sources.yml created
- ✓ stg_products.sql created
- ✓ schema.yml created

- ✓ Documentation added
- ✓ Data quality tests added
- ✓ dbt run completed successfully
- ✓ dbt test completed successfully
- ✓ ANALYTICS.STG_PRODUCTS created in Snowflake

STEP 6 – BUILD THE INTERMEDIATE LAYER

Goal

We'll create a model that enriches the staging data with business-friendly attributes such as:

- Price Segment
- Rating Category
- Review Volume
- Price Band

Project Structure

Your dbt project should now look like:

```
dbt_project/
├── models/
│   ├── staging/
│   │   ├── sources.yml
│   │   ├── schema.yml
│   │   └── stg_products.sql
│   ├── intermediate/
│   │   ├── int_products.sql
│   │   └── schema.yml
│   └── marts/
```

Step 1. Create int_products.sql

Create:

```
models/  
└─ intermediate/  
   └─ int_products.sql
```

Paste the following:

```
SELECT  
  
    id,  
    title,  
    description,  
    category,  
    image,  
    price,  
    rating_rate,  
    rating_count,  
  
    -- Price Segmentation  
    CASE  
        WHEN price < 50 THEN 'Budget'  
        WHEN price < 100 THEN 'Standard'  
        ELSE 'Premium'  
    END AS price_segment,  
  
    -- Rating Classification  
    CASE  
        WHEN rating_rate >= 4.5 THEN 'Excellent'  
        WHEN rating_rate >= 4.0 THEN 'Good'  
        WHEN rating_rate >= 3.0 THEN 'Average'  
        ELSE 'Poor'  
    END AS rating_category,  
  
    -- Review Volume  
    CASE  
        WHEN rating_count >= 300 THEN 'Very High'  
        WHEN rating_count >= 150 THEN 'High'  
        WHEN rating_count >= 50 THEN 'Medium'  
        ELSE 'Low'  
    END AS review_volume,  
  
    -- Price Band  
    CASE  
        WHEN price < 25 THEN '0 - 25'  
        WHEN price < 50 THEN '25 - 50'  
        WHEN price < 100 THEN '50 - 100'  
        WHEN price < 250 THEN '100 - 250'
```

```
        ELSE '250+'
      END AS price_band
FROM {{ ref('stg_products') }}
```

Step 2. Create schema.yml

Create:

```
models/
└─ intermediate/
   └─ schema.yml
```

Paste:

```
version: 2

models:
  - name: int_products
    description: >
      Intermediate business model that enriches staged product data with
      reusable business classifications such as price segments, rating
      categories, review volume, and price bands.
    columns:

      - name: id
        description: Unique product identifier.
        tests:
          - unique
          - not_null

      - name: title
        description: Product title.

      - name: category
        description: Product category.

      - name: price
        description: Product selling price.

      - name: price_segment
        description: Business classification based on product price.
        tests:
          - accepted_values:
              values:
```

- Budget
 - Standard
 - Premium
-
- name: rating_category
description: Rating quality classification.
 - name: review_volume
description: Review count grouping.
 - name: price_band
description: Price range bucket.

Step 3. Run dbt

```
dbt run --profiles-dir .
```

Expected output:

```
2 of 2 START sql view model ANALYTICS.int_products
2 of 2 OK created sql view model ANALYTICS.int_products
```

Step 4. Run Tests

```
dbt test --profiles-dir .
```

Expected:

```
PASS unique_int_products_id
PASS not_null_int_products_id
PASS accepted_values_int_products_price_segment
```

Step 5. Verify in Snowflake

```
SELECT
  title,
  category,
  price,
  price_segment,
  rating_rate,
  rating_category,
  review_volume,
```

```
price_band
FROM ANALYTICS.INT_PRODUCTS
LIMIT 10;
```

Example output:

Title	Price	Price Band	Segment	Rating	Rating Category	Review Volume
Backpack	109.95	Premium	3.9	Average	Medium	100 - 250
Shirt	22.30	Budget	4.7	Excellent	High	0 - 25

Why do we need an Intermediate Layer?

Instead of putting all business logic directly into reporting models, we centralize it here.

For example:

- **Staging Layer** → Clean the raw data.
- **Intermediate Layer** → Apply reusable business logic.
- **Gold Layer** → Aggregate data for dashboards and reporting.

This separation makes the project easier to maintain and follows dbt best practices.

Step 6 Checklist

- ✓ Created int_products.sql
- ✓ Created schema.yml
- ✓ Added business logic with reusable classifications
- ✓ Added model documentation
- ✓ Added data quality tests
- ✓ Ran dbt run successfully

- ✓ Ran dbt test successfully
- ✓ Verified ANALYTICS.INT_PRODUCTS in Snowflake

STEP 7 – BUILD THE GOLD LAYER

Goal

Create three reporting models:

```
models/  
└─ marts/  
    ├── gold_category_summary.sql  
    ├── gold_product_ratings.sql  
    ├── gold_price_segments.sql  
    └─ schema.yml
```

Gold Model 1: Category Summary

Create gold_category_summary.sql

```
SELECT  
  
    category,  
  
    COUNT(*) AS total_products,  
  
    ROUND(AVG(price), 2) AS average_price,  
  
    ROUND(AVG(rating_rate), 2) AS average_rating,  
  
    SUM(rating_count) AS total_reviews,  
  
    MIN(price) AS minimum_price,  
  
    MAX(price) AS maximum_price  
  
FROM {{ ref('int_products') }}  
  
GROUP BY category
```

Expected Output

Category	Total	Products	Avg Price	Avg Rating	Total Reviews
Electronics	6	332.50	4.33	2,145	
Jewelry	4	211.87	4.12	875	

Gold Model 2: Product Ratings

Create gold_product_ratings.sql

```
SELECT
    id,
    title,
    category,
    price,
    rating_rate,
    rating_count,
    rating_category,
    review_volume,
    DENSE_RANK() OVER (
        ORDER BY rating_rate DESC,
                rating_count DESC
    ) AS product_rank
FROM {{ ref('int_products') }}
```

Expected Output

Rank	Product	Rating	Reviews
1	Product A	4.9	550
2	Product B	4.8	430

Gold Model 3: Price Segments

Create gold_price_segments.sql

```
SELECT
    price_segment,
    price_band,
    COUNT(*) AS total_products,
```

```

ROUND(AVG(price), 2) AS average_price,

ROUND(AVG(rating_rate), 2) AS average_rating

FROM {{ ref('int_products') }}

GROUP BY

    price_segment,
    price_band

```

Expected Output

Segment	Band	Products	Avg Price
Budget	0-25	8	18.50
Standard	50-100	5	74.20
Premium	100-250	7	168.10

Create schema.yml

Inside models/marts/, create a schema.yml file.

```

version: 2

models:
  - name: gold_category_summary
    description: >
      Gold reporting model that summarizes product performance by category
    .
      Used by Power BI for category-level KPIs and visualizations.
    columns:
      - name: category
        description: Product category.
        tests:
          - unique
          - not_null

      - name: total_products
        description: Number of products in the category.

      - name: average_price
        description: Average product price.

```



```

- name: average_rating
  description: Average customer rating.

- name: total_reviews
  description: Total number of customer reviews.

- name: gold_product_ratings
  description: >
    Gold reporting model containing ranked products based on customer
    ratings and review counts.
  columns:

    - name: product_rank
      description: Product ranking by rating and review count.

    - name: rating_rate
      description: Average customer rating.

    - name: rating_count
      description: Number of customer reviews.

- name: gold_price_segments
  description: >
    Gold reporting model summarizing pricing segments for analytical
    dashboards.
  columns:

    - name: price_segment
      description: Product price classification.

    - name: price_band
      description: Detailed price range.

    - name: total_products
      description: Number of products in each price segment.

```

Run dbt

Execute:

```
dbt run --profiles-dir .
```

Expected output:

```
3 of 5 OK created table model ANALYTICS.gold_category_summary
```

```
4 of 5 OK created table model ANALYTICS.gold_product_ratings
5 of 5 OK created table model ANALYTICS.gold_price_segments

Done.
```

Run Tests

```
dbt test --profiles-dir .
```

All tests should pass.

Verify in Snowflake

Run the following queries:

```
SELECT * FROM ANALYTICS.GOLD_CATEGORY_SUMMARY;
SELECT * FROM ANALYTICS.GOLD_PRODUCT_RATINGS;
SELECT * FROM ANALYTICS.GOLD_PRICE_SEGMENTS;
```

Updated dbt Project Structure

```
dbt_project/
├── models/
├── staging/
│   ├── sources.yml
│   ├── schema.yml
│   └── stg_products.sql
├── intermediate/
│   ├── int_products.sql
│   └── schema.yml
└── marts/
    ├── gold_category_summary.sql
    ├── gold_product_ratings.sql
    ├── gold_price_segments.sql
    └── schema.yml
```

Generate Documentation

Now that all your models are in place, generate the dbt documentation:

```
dbt docs generate --profiles-dir .
```

Serve the documentation locally:

```
dbt docs serve --profiles-dir .
```

Verify that the documentation includes:

- ✓ Model descriptions
- ✓ Column descriptions
- ✓ Source documentation
- ✓ Data lineage graph showing RAW.PRODUCTS → STG_PRODUCTS → INT_PRODUCTS → GOLD_*
- ✓ Test results

STEP 8 – ORCHESTRATE THE PIPELINE WITH AIRFLOW

Goal

Create an Airflow DAG that automatically executes the pipeline.

The workflow will be:

```
extract_api
  |
  ▼
load_snowflake
  |
  ▼
dbt_run
  |
  ▼
dbt_test
```

Step 1. Folder Structure

Your project should look like this:

```
Capstone_Project/
|
├── airflow/
|   └── dags/
```

```
├── ecommerce_pipeline.py
├── logs/
├── plugins/
├── config/
├── scripts/
│   ├── extract.py
│   └── load_snowflake.py
├── dbt_project/
├── docker-compose.yml
└── .env
```

Step 2. Mount the Project

Since you're using Docker on Windows, your docker-compose.yml should already mount your project.

It should contain something similar to:

```
volumes:
  - ./airflow/dags:/opt/airflow/dags
  - ./:/opt/airflow/project
```

This means inside the container:

```
/opt/airflow/project
```

is your entire project.

Therefore,

```
scripts/extract.py
```

becomes

```
/opt/airflow/project/scripts/extract.py
```

inside Airflow.

Step 3. Install Required Packages in Airflow

Since Airflow runs in Docker, install packages inside the container.

Add to your Airflow image or requirements.txt:

```
snowflake-connector-python
dbt-core
dbt-snowflake
python-dotenv
pandas
requests
```

Rebuild the containers:

```
docker compose down
docker compose build
docker compose up -d
```

Step 4. Create the DAG

Create:

```
airflow/dags/ecommerce_pipeline.py
```

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {
    "owner": "airflow",
    "depends_on_past": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=2),
}

with DAG(
    dag_id="ecommerce_pipeline",
    description="End-to-End Ecommerce Data Pipeline",
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    default_args=default_args,
```

```

tags=["capstone", "snowflake", "dbt"],
) as dag:

    extract_api = BashOperator(
        task_id="extract_api",
        bash_command="""
        cd /opt/airflow/project &&
        python scripts/extract.py
        """
    )

    load_snowflake = BashOperator(
        task_id="load_snowflake",
        bash_command="""
        cd /opt/airflow/project &&
        python scripts/load_snowflake.py
        """
    )

    dbt_run = BashOperator(
        task_id="dbt_run",
        bash_command="""
        cd /opt/airflow/project/dbt_project &&
        dbt run --profiles-dir .
        """
    )

    dbt_test = BashOperator(
        task_id="dbt_test",
        bash_command="""
        cd /opt/airflow/project/dbt_project &&
        dbt test --profiles-dir .
        """
    )

    extract_api >> load_snowflake >> dbt_run >> dbt_test

```

Step 5. Verify the DAG

Restart Airflow:

```
docker compose restart
```

Open:

You should see:

```
ecommerce_pipeline
```

Step 6. Trigger the DAG

Click

► **Trigger** DAG

The Graph View should show:

```
graph TD
    extract_api --> load_snowflake
    load_snowflake --> dbt_run
    dbt_run --> dbt_test
```

After a successful run:

```
■ extract_api
■ load_snowflake
■ dbt_run
■ dbt_test
```

Step 7. Verify the Results

Each run should:

- Download the latest data from the API.
- Overwrite the contents of RAW.PRODUCTS.

- Rebuild all dbt models.
- Run all dbt tests.

You can verify the results in Snowflake:

```
SELECT COUNT(*) FROM RAW.PRODUCTS;  
  
SELECT COUNT(*) FROM ANALYTICS.GOLD_CATEGORY_SUMMARY;  
  
SELECT COUNT(*) FROM ANALYTICS.GOLD_PRODUCT_RATINGS;  
  
SELECT COUNT(*) FROM ANALYTICS.GOLD_PRICE_SEGMENTS;
```

One important improvement

I recommend **not** using BashOperator in the final version of your project. Since you already have a Docker-based Airflow setup, a more professional approach is to use:

- PythonOperator for extract.py
- PythonOperator for load_snowflake.py
- BashOperator only for dbt run
- BashOperator only for dbt test

This provides better logging, easier debugging, and cleaner task execution while still running entirely inside Docker—**no Windows Python installation or virtual environment is required.**

STEP 9 – CI/CD WITH GITHUB ACTIONS

Goal

Every time you push to GitHub, GitHub Actions should automatically:

- Install dependencies
- Verify Python scripts compile
- Verify the dbt project is valid

- (Optionally) run dbt when secrets are configured

Important: Your Snowflake credentials should **never** be stored in the repository. Use GitHub Secrets if you want the workflow to connect to Snowflake.

Project Structure

```
Capstone_Project/  
├── .github/  
│   └── workflows/  
│       └── pipeline.yml  
├── scripts/  
├── dbt_project/  
├── airflow/  
└── ...
```

Step 1. Create the Workflow

Create:

```
.github/workflows/pipeline.yml
```

Paste the following:

```
name: Data Pipeline CI  
  
on:  
  push:  
    branches:  
      - main  
      - master  
  
  pull_request:  
    branches:  
      - main  
      - master  
  
jobs:  
  
  validate:
```

```

runs-on: ubuntu-latest

steps:
  - name: Checkout Repository
    uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.12"

  - name: Install Python Packages
    run: |
      python -m pip install --upgrade pip
      pip install requests pandas snowflake-connector-python python-
dotenv
  - name: Verify Python Scripts
    run: |
      python -m py_compile scripts/extract.py
      python -m py_compile scripts/load_snowflake.py
  - name: Install dbt
    run: |
      pip install dbt-core dbt-snowflake
  - name: Verify dbt Installation
    run: dbt --version

```

This workflow validates that:

- Python scripts have no syntax errors.
- dbt installs correctly.
- Your repository is structurally sound.

It doesn't require Snowflake credentials.

Step 2. Optional Snowflake Validation

If you want GitHub Actions to run dbt debug, dbt run, and dbt test, add these repository secrets in GitHub:

- SNOWFLAKE_ACCOUNT
- SNOWFLAKE_USER

- SNOWFLAKE_PASSWORD
- SNOWFLAKE_ROLE
- SNOWFLAKE_DATABASE
- SNOWFLAKE_SCHEMA
- SNOWFLAKE_WAREHOUSE

Then extend the workflow:

```
env:
  SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }
  SNOWFLAKE_USER: ${ secrets.SNOWFLAKE_USER }
  SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }
  SNOWFLAKE_ROLE: ${ secrets.SNOWFLAKE_ROLE }
  SNOWFLAKE_DATABASE: ${ secrets.SNOWFLAKE_DATABASE }
  SNOWFLAKE_SCHEMA: ${ secrets.SNOWFLAKE_SCHEMA }
  SNOWFLAKE_WAREHOUSE: ${ secrets.SNOWFLAKE_WAREHOUSE }
```

And add these steps:

```
- name: dbt Debug
  working-directory: dbt_project
  run: dbt debug --profiles-dir .

- name: dbt Run
  working-directory: dbt_project
  run: dbt run --profiles-dir .

- name: dbt Test
  working-directory: dbt_project
  run: dbt test --profiles-dir .
```

Step 3. Push to GitHub

```
git add .

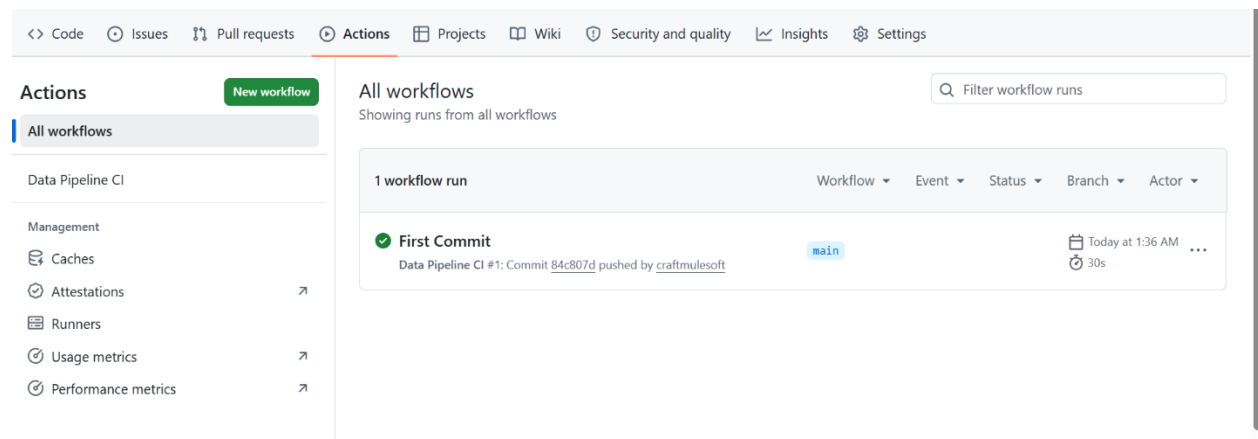
git commit -m "Add GitHub Actions CI pipeline"

git push origin main
```

After pushing, go to your GitHub repository.

Click:

Actions



You should see:

✓ Data Pipeline CI

If all steps succeed, your workflow is correctly configured.

Step 9 Checklist

- ✓ Created GitHub Actions workflow
- ✓ Validated Python scripts
- ✓ Installed dbt in CI
- ✓ (Optional) Configured GitHub Secrets for Snowflake
- ✓ Workflow runs successfully on every push

STEP 10 – PROFESSIONAL DOCUMENTATION

Goal

Create professional documentation including:

- README.md
- Architecture Diagram
- Project Structure

- Technology Stack
- Data Flow
- Setup Instructions
- Screenshots
- Future Improvements

Folder Structure

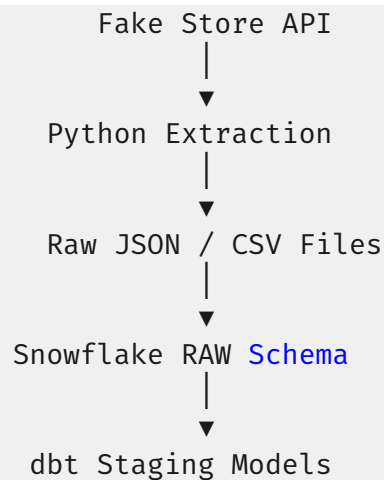
Your project should now look like this:

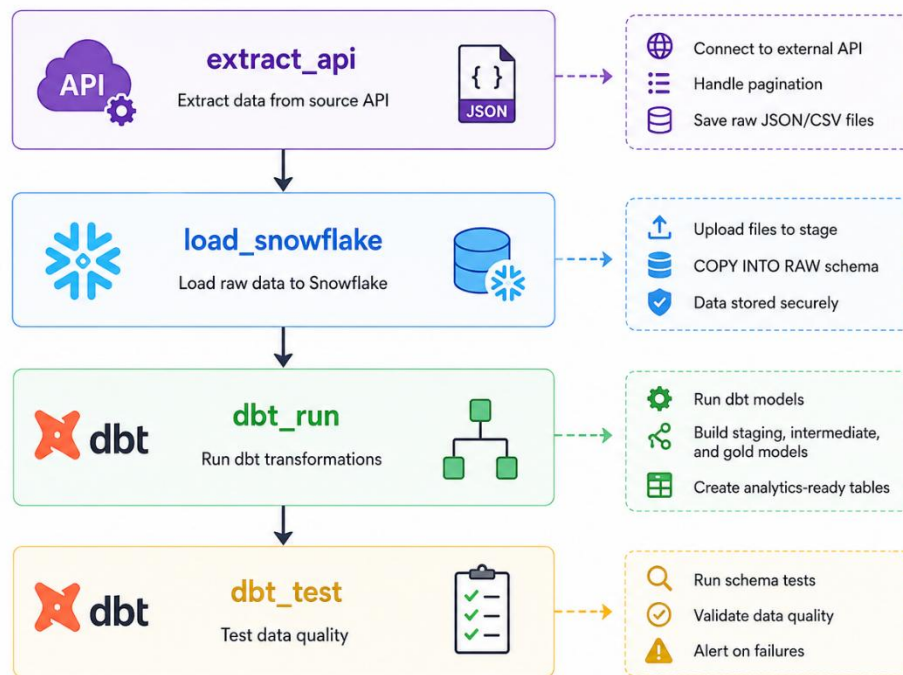
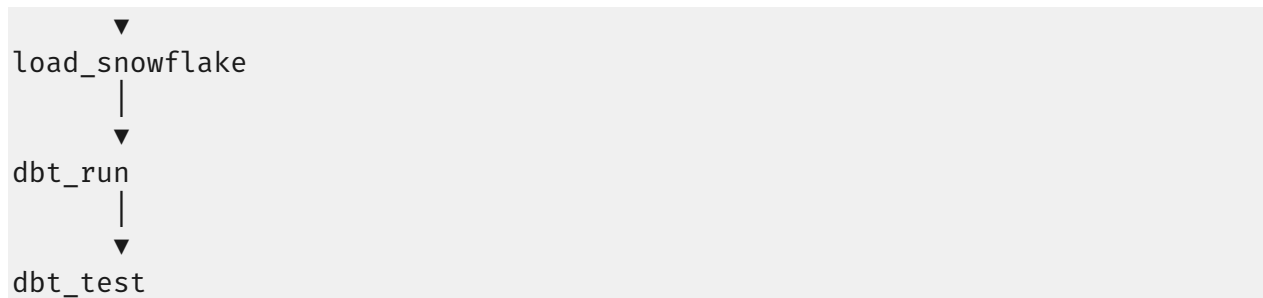
```
Capstone_Project/  
├── docs/  
│   ├── architecture.png  
│   ├── airflow_pipeline.png  
│   ├── dbt_lineage.png  
│   ├── dashboard_overview.png  
│   └── snowflake_schema.png  
├── README.md  
└── ...
```

1. Create the Architecture Diagram

You already created one similar during our Airflow project.

The final architecture should look like this.





Save as

```
docs/airflow_pipeline.png
```

3. dbt Lineage Screenshot

Run

```
dbt docs generate --profiles-dir .
```

Then

```
dbt docs serve --profiles-dir .
```

Open

```
http://localhost:8081
```

Take a screenshot of the lineage graph.

Save as

```
docs/dbt_lineage.png
```

7. Final Repository Structure

```
Capstone_Project/  
├── airflow/  
├── data/  
├── dbt_project/  
├── docs/  
│   ├── architecture.png  
│   ├── airflow_pipeline.png  
│   ├── dbt_lineage.png  
│   ├── dashboard_overview.png  
│   └── snowflake_schema.png  
├── powerbi/  
│   └── Ecommerce Dashboard.pbix  
├── scripts/  
├── sql/  
├── tests/  
├── .github/  
├── README.md  
└── docker-compose.yml
```

complete production-quality docker-compose.yml that is suitable for your capstone project.

Features:

- ✓ Windows compatible

- ✓ Docker Desktop compatible
- ✓ LocalExecutor
- ✓ PostgreSQL backend
- ✓ Custom Airflow image (using airflow/Dockerfile)
- ✓ Single command startup (docker compose up -d --build)
- ✓ Automatic database initialization
- ✓ Automatic admin user creation
- ✓ Health checks
- ✓ Mounts your entire project so Airflow can run your Python scripts and dbt project

```
version: "3.8"

x-airflow-common: &airflow-common
  build:
    context: ./airflow

  env_file:
    - .env

  environment:
    AIRFLOW_UID: 50000

    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__CORE__LOAD_EXAMPLES: "False"
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: "True"

    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:air
flow@postgres/airflow

  volumes:
    - ./airflow/dags:/opt/airflow/dags
    - ./airflow/logs:/opt/airflow/logs
    - ./airflow/plugins:/opt/airflow/plugins

# Entire project
- ./:/opt/airflow/project
```

```
user: "${AIRFLOW_UID:-50000}:0"

services:

  postgres:
    image: postgres:16

    container_name: airflow-postgres

    restart: unless-stopped

    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow

    ports:
      - "5432:5432"

    volumes:
      - postgres-db-volume:/var/lib/postgresql/data

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U airflow"]
      interval: 5s
      timeout: 5s
      retries: 10

  airflow-init:

    <<: *airflow-common

    container_name: airflow-init

    depends_on:
      postgres:
        condition: service_healthy

    restart: "no"

    command: >
      bash -c "
      airflow db migrate &&
      airflow users create
      --username admin
      --firstname Admin
```

```
--lastname User
--role Admin
--email admin@example.com
--password admin || true
"

airflow-webserver:

  <<: *airflow-common

  container_name: airflow-webserver

  depends_on:
    postgres:
      condition: service_healthy

    airflow-init:
      condition: service_completed_successfully

  restart: unless-stopped

  ports:
    - "8080:8080"

  command: airflow webserver

airflow-scheduler:

  <<: *airflow-common

  container_name: airflow-scheduler

  depends_on:
    postgres:
      condition: service_healthy

    airflow-init:
      condition: service_completed_successfully

  restart: unless-stopped

  command: airflow scheduler

volumes:
  postgres-db-volume:
```

Corresponding airflow/Dockerfile

```
FROM apache/airflow:2.11.0-python3.12

USER root

RUN apt-get update && \
    apt-get install -y git gcc g++ && \
    apt-get clean

USER airflow

COPY requirements.txt /requirements.txt

RUN pip install --no-cache-dir -r /requirements.txt
```

airflow/requirements.txt

```
pandas
requests
snowflake-connector-python
python-dotenv

dbt-core
dbt-snowflake
```

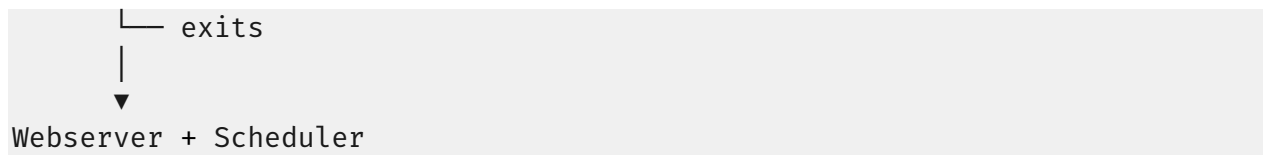
Start Everything

Now, the only command you need is:

```
docker compose up -d --build
```

The startup sequence will be:

```
PostgreSQL
|
▼
Health Check
|
▼
airflow-init
|
├─ airflow db migrate
└─ create admin user (if needed)
```



Then open:

- **Airflow UI:** <http://localhost:8080>

Login:

- **Username:** admin
- **Password:** admin

Below is a **production-ready pipeline.yml** for your capstone project. It will:

- ✓ Run on every push and pull request
- ✓ Install Python dependencies
- ✓ Install dbt
- ✓ Validate Python scripts
- ✓ Connect to Snowflake using **GitHub Secrets**
- ✓ Run dbt debug
- ✓ Run dbt deps
- ✓ Run dbt run
- ✓ Run dbt test

Prerequisites

Your GitHub repository must have the following **Repository Secrets** configured:

- SNOWFLAKE_ACCOUNT
- SNOWFLAKE_USER

- SNOWFLAKE_PASSWORD
- SNOWFLAKE_ROLE
- SNOWFLAKE_WAREHOUSE
- SNOWFLAKE_DATABASE
- SNOWFLAKE_SCHEMA

.github/workflows/pipeline.yml

```
name: Data Pipeline CI/CD

on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main

jobs:
  validate-and-test:

    runs-on: ubuntu-latest

    env:
      SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }
      SNOWFLAKE_USER: ${ secrets.SNOWFLAKE_USER }
      SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }
      SNOWFLAKE_ROLE: ${ secrets.SNOWFLAKE_ROLE }
      SNOWFLAKE_WAREHOUSE: ${ secrets.SNOWFLAKE_WAREHOUSE }
      SNOWFLAKE_DATABASE: ${ secrets.SNOWFLAKE_DATABASE }
      SNOWFLAKE_SCHEMA: ${ secrets.SNOWFLAKE_SCHEMA }

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
```

```

- name: Upgrade pip
  run: |
    python -m pip install --upgrade pip
- name: Install Project Dependencies
  run: |
    pip install -r requirements.txt
- name: Install dbt
  run: |
    pip install dbt-core dbt-snowflake
- name: Verify Python Scripts
  run: |
    python -m py_compile scripts/extract.py
    python -m py_compile scripts/load_snowflake.py
- name: Verify dbt Installation
  run: |
    dbt --version
- name: Install dbt Packages
  working-directory: dbt_project
  run: |
    dbt deps --profiles-dir .
- name: dbt Debug
  working-directory: dbt_project
  run: |
    dbt debug --profiles-dir .
- name: dbt Run
  working-directory: dbt_project
  run: |
    dbt run --profiles-dir .
- name: dbt Test
  working-directory: dbt_project
  run: |
    dbt test --profiles-dir .

```

Your dbt_project/profiles.yml

To make this pipeline work, your profiles.yml **must** use environment variables rather than hard-coded credentials.

```

dbt_project:
  target: dev

outputs:
  dev:
    type: snowflake

```

```
account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
user: "{{ env_var('SNOWFLAKE_USER') }}"
password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"

role: "{{ env_var('SNOWFLAKE_ROLE') }}"

warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"
database: "{{ env_var('SNOWFLAKE_DATABASE') }}"
schema: "{{ env_var('SNOWFLAKE_SCHEMA') }}"

threads: 4

client_session_keep_alive: false
```

requirements.txt

Your root requirements.txt should contain all the required packages so the GitHub Actions workflow can install everything with a single command:

```
requests
pandas
snowflake-connector-python
python-dotenv

dbt-core
dbt-snowflake
```

GitHub Repository Secrets

Go to:

GitHub → Your Repository → Settings → Secrets and variables → Actions

Create these secrets:

Secret Name	Example Value
SNOWFLAKE_ACCOUNT	abc12345.us-east-1
SNOWFLAKE_USER	your_username
SNOWFLAKE_PASSWORD	your_password
SNOWFLAKE_ROLE	ACCOUNTADMIN
SNOWFLAKE_WAREHOUSE	COMPUTE_WH
SNOWFLAKE_DATABASE	ECOMMERCE_DB
SNOWFLAKE_SCHEMA	ANALYTICS

Since your dbt models are materialized into the ANALYTICS schema, the SNOWFLAKE_SCHEMA secret should be ANALYTICS, even though your source data lives in the RAW schema.

STEP 11 - POWER BI DASHBOARD ROADMAP

We'll build **5 professional dashboard pages**.

Page 1 → Executive Overview
Page 2 → Category Analysis
Page 3 → Product Performance
Page 4 → Pricing Analysis
Page 5 → Data Quality & Pipeline Monitoring

Each page has a clear business purpose.

Dashboard Data Model

Connect Power BI to these tables in Snowflake:

ANALYTICS.GOLD_CATEGORY_SUMMARY
ANALYTICS.GOLD_PRODUCT_RATINGS
ANALYTICS.GOLD_PRICE_SEGMENTS

STEP 1 — Connect Power BI to Snowflake

Open Power BI Desktop.

Select:

Get Data



Snowflake

Enter:

Server:

<your_account>.snowflakecomputing.com

Warehouse:

COMPUTE_WH

Database:

ECOMMERCE_DB

Schema:

ANALYTICS

Load:

GOLD_CATEGORY_SUMMARY

GOLD_PRODUCT_RATINGS

GOLD_PRICE_SEGMENTS

PAGE 1 — Executive Overview

This page answers:

"How is the product catalog performing overall?"

KPI Cards

Create five cards.

Card 1

Total Products

Measure

Total Products =

SUM('GOLD_CATEGORY_SUMMARY'[TOTAL_PRODUCTS])

Card 2

Average Product Price

Average Price =

AVERAGE('GOLD_CATEGORY_SUMMARY'[AVERAGE_PRICE])

Card 3

Average Rating

Average Rating =

AVERAGE('GOLD_CATEGORY_SUMMARY'[AVERAGE_RATING])

Card 4

Total Reviews

Total Reviews =

SUM('GOLD_CATEGORY_SUMMARY'[TOTAL_REVIEWS])

Card 5

Categories

Categories =

DISTINCTCOUNT('GOLD_CATEGORY_SUMMARY'[CATEGORY])

Charts

Donut Chart

Legend

CATEGORY

Values

TOTAL_PRODUCTS

Shows distribution of products.

Column Chart

Axis

CATEGORY

Values

AVERAGE_PRICE

Shows average selling price by category.

Bar Chart

Axis

CATEGORY

Values

AVERAGE_RATING

Shows category ratings.

Table

Category

Total Products

Average Price

Average Rating

Total Reviews

Slicers

Category

Layout

KPI KPI KPI KPI KPI

Donut Column Chart

Bar Chart Summary Table

Page 2 – Category Analysis

Goal

Create a page that provides detailed insights into each product category.

Page Layout

CATEGORY ANALYSIS

Slicer (Category)

KPI KPI KPI KPI

Clustered Column Horizontal Bar

Treemap Detailed Table

Step 1. Duplicate the Theme

Use the same theme colors as Page 1 for consistency.

Background:

#F8FAFC

Step 2. Add a Category Slicer

Insert a **Slicer**.

Field:

CATEGORY

Enable:

- Search
- Single Select = Off

- Multi Select = On

This slicer should filter the entire page.

Step 3. Create KPI Cards

KPI 1 — Total Products

Total Products =

SUM('GOLD_CATEGORY_SUMMARY'[TOTAL_PRODUCTS])

KPI 2 — Average Price

Average Category Price =

AVERAGE('GOLD_CATEGORY_SUMMARY'[AVERAGE_PRICE])

KPI 3 — Average Rating

Average Category Rating =

AVERAGE('GOLD_CATEGORY_SUMMARY'[AVERAGE_RATING])

KPI 4 — Total Reviews

Total Reviews =

SUM('GOLD_CATEGORY_SUMMARY'[TOTAL_REVIEWS])

Arrange the KPI cards horizontally.

Products

Average Price

Average Rating

Total Reviews

Step 4. Clustered Column Chart

Title

Average Price by Category

Fields

Axis

CATEGORY

Values

AVERAGE_PRICE

Sort

Descending

Turn on:

- Data Labels
 - Gridlines
 - Tooltip
-

Step 5. Horizontal Bar Chart

Title

Average Rating by Category

Axis

CATEGORY

Values

AVERAGE_RATING

Conditional Formatting:

- Highest Rating → Green
- Lowest Rating → Red

Step 6. Treemap

Title

Product Distribution by Category

Group

CATEGORY

Values

TOTAL_PRODUCTS

This quickly shows the size of each category.

Step 7. Detailed Table

Columns:

CATEGORY

TOTAL_PRODUCTS

AVERAGE_PRICE

AVERAGE_RATING

TOTAL_REVIEWS

MINIMUM_PRICE

MAXIMUM_PRICE

Enable:

- Totals
 - Search
 - Alternate Row Colors
-

Step 8. Add Drillthrough

Create a Drillthrough page.

Drag:

CATEGORY

into the Drillthrough Filters pane.

Now, users can:

Right-click a category →

Drillthrough

↓

Category Details

This creates a more interactive report.

Step 9. Dynamic Page Title

Create this measure:

Selected Category =

IF(

HASONEVALUE('GOLD_CATEGORY_SUMMARY'[CATEGORY]),

SELECTEDVALUE('GOLD_CATEGORY_SUMMARY'[CATEGORY]),

"All Categories"
)

Then create:

Category Page Title =
"Category Analysis - " & [Selected Category]

Use this measure as the visual title (via conditional formatting using a field value) so the page title updates based on the slicer selection.

Final Layout

Category Slicer

Products Avg Price Avg Rating Reviews

Average Price Average Rating

Treemap Detailed Table

Insights This Page Provides

A business user can answer:

- Which category has the highest average price?
 - Which category has the highest average rating?
 - Which category receives the most reviews?
 - Which categories have the most products?
 - What are the minimum and maximum prices within each category?
-

Page 2 Checklist

- ✓ Category slicer
- ✓ 4 KPI cards
- ✓ Average price chart
- ✓ Average rating chart
- ✓ Treemap
- ✓ Detailed category table
- ✓ Drillthrough enabled
- ✓ Dynamic page title

Page 3 – Product Performance

Business Question

This page answers:

- Which products are the highest rated?
- Which products receive the most reviews?
- Which products generate the highest prices?
- Which products are Premium, Standard, or Budget?

- Which products should customers or management focus on?

Page Layout

PRODUCT PERFORMANCE

Category Slicer Price Segment Slicer

Highest Rated Avg Rating Total Reviews Products

Top Rated Products Most Reviewed Products

Scatter Plot Product Ranking Table

Step 1 – Add Slicers

Slicer 1

Field

CATEGORY

Slicer 2

Field

PRICE_SEGMENT

Optional Slicer

Field

RATING_CATEGORY

Arrange them across the top of the page.

Step 2 – Create KPI Measures

Total Products

Total Products =
COUNTROWS('GOLD_PRODUCT_RATINGS')

Average Rating

Average Rating =
AVERAGE('GOLD_PRODUCT_RATINGS'[RATING_RATE])

Total Reviews

Total Reviews =
SUM('GOLD_PRODUCT_RATINGS'[RATING_COUNT])

Highest Product Rating

Highest Rating =
MAX('GOLD_PRODUCT_RATINGS'[RATING_RATE])

Arrange the KPI cards:

Highest Rating

Average Rating

Total Reviews

Products

Step 3 – Top Rated Products

Insert a **Clustered Bar Chart**.

Title

Top 10 Highest Rated Products

Axis

TITLE

Values

RATING_RATE

Filter

Top N

10

By

RATING_RATE

Sort descending.

Turn on:

- Data Labels
- Tooltips

Step 4 – Most Reviewed Products

Insert another **Bar Chart**.

Title

Top 10 Most Reviewed Products

Axis

TITLE

Values

RATING_COUNT

Top N

10

Step 5 – Scatter Chart

This is one of the most informative visuals.

Title

Price vs Rating

X Axis

PRICE

Y Axis

RATING_RATE

Size

RATING_COUNT

Legend

PRICE_SEGMENT

Details

TITLE

This allows users to quickly identify:

- High-priced, highly-rated products
- Low-priced products with excellent ratings
- Products with many reviews

Step 6 – Product Ranking Table

Create a **Table**.

Columns

PRODUCT_RANK

TITLE

CATEGORY

PRICE

PRICE_SEGMENT

RATING_RATE

RATING_COUNT

RATING_CATEGORY

REVIEW_VOLUME

Enable:

- Search
- Sorting
- Alternate Row Colors

Sort by:

PRODUCT_RANK

Ascending.

Step 7 – Rich Tooltips

Create a new page.

Rename

Product Tooltip

Page Information

Tooltip = On

Canvas Size

Tooltip

Add:

- Product Name
- Category
- Price
- Rating
- Reviews
- Price Segment
- Rating Category

Assign this page as the tooltip for the bar and scatter charts.

Step 8 – Drillthrough Page

Create a new page.

Rename

Product Details

Add Drillthrough Field

TITLE

Add visuals:

- Product Information
- Rating
- Review Count
- Price
- Category
- Price Segment

Now users can right-click a product and navigate to detailed information.

Step 9 – Dynamic Page Title

Create a measure:

Selected Product Category =

```
IF(
    HASONEVALUE('GOLD_PRODUCT_RATINGS'[CATEGORY]),
    SELECTEDVALUE('GOLD_PRODUCT_RATINGS'[CATEGORY]),
    "All Categories"
)
```

Then:

Product Page Title =

"Product Performance - " &
[Selected Product Category]

Use this measure for the page title through conditional formatting.

Final Layout

Category	Price Segment	Rating Category
----------	---------------	-----------------

Highest Rating	Avg Rating	Reviews	Products
----------------	------------	---------	----------

Top Rated	Most Reviewed
-----------	---------------

Scatter Plot Ranking Table

Insights This Page Provides

Business users can identify:

- ★ Highest-rated products
- 🛒 Most-reviewed products
- 💰 Premium products with excellent ratings
- 📉 Expensive products with poor ratings
- 🎯 Products that are popular but underperforming
- 📊 The relationship between price, customer ratings, and review volume

Page 3 Checklist

- ✓ Category slicer
- ✓ Price segment slicer
- ✓ Rating category slicer
- ✓ KPI cards
- ✓ Top 10 highest-rated products
- ✓ Top 10 most-reviewed products
- ✓ Price vs. rating scatter plot
- ✓ Product ranking table
- ✓ Rich tooltips

✓ Drillthrough page

✓ Dynamic page titles

Page 4 – Pricing Analysis

Business Questions

This page answers:

- How are products distributed across price segments?
- Which price bands have the most products?
- Do expensive products receive better ratings?
- Which categories are premium?
- How does pricing vary across the catalog?

Page Layout

PRICING ANALYSIS

Price Segment Slicer Category Slicer

Average Price Products Avg Rating Premium Products

Price Segment Chart Price Band Chart

Scatter Plot Matrix

Treemap Donut Chart

Step 1 – Add Slicers

Slicer 1

Field

PRICE_SEGMENT

Slicer 2

Field

CATEGORY

Step 2 – Create KPI Measures

Average Product Price

Average Price =

AVERAGE('GOLD_PRODUCT_RATINGS'[PRICE])

Total Products

Total Products =

```
COUNTROWS('GOLD_PRODUCT_RATINGS')
```

Average Rating

Average Rating =

```
AVERAGE('GOLD_PRODUCT_RATINGS'[RATING_RATE])
```

Premium Products

Premium Products =

```
CALCULATE(  
    COUNTROWS('GOLD_PRODUCT_RATINGS'),  
    'GOLD_PRODUCT_RATINGS'[PRICE_SEGMENT] = "Premium"  
)
```

Arrange the KPI cards:

Average Price

Products

Average Rating

Premium Products

Step 3 – Column Chart

Title

Products by Price Segment

Axis

PRICE_SEGMENT

Values

TOTAL_PRODUCTS

Use the **GOLD_PRICE_SEGMENTS** table.

Sort:

Budget

Standard

Premium

Step 4 – Bar Chart

Title

Products by Price Band

Axis

PRICE_BAND

Values

TOTAL_PRODUCTS

Sort:

0-25

25-50

50-100

100-250

250+

Step 5 – Scatter Plot

This is one of the most valuable visuals.

Title

Price vs Customer Rating

X Axis

PRICE

Y Axis

RATING_RATE

Bubble Size

RATING_COUNT

Legend

PRICE_SEGMENT

Details

TITLE

This shows:

- Expensive products with excellent ratings
 - Cheap products with excellent ratings
 - Products with many reviews
-

Step 6 – Matrix

Title

Category Pricing Summary

Rows

CATEGORY

Columns

PRICE_SEGMENT

Values

Average Price

Average Rating

Enable:

- Conditional Formatting
 - Totals
 - Expand/Collapse
-

Step 7 – Treemap

Title

Revenue Distribution by Price Segment

Group

PRICE_SEGMENT

Values

TOTAL_PRODUCTS

Step 8 – Donut Chart

Title

Product Mix by Price Segment

Legend

PRICE_SEGMENT

Values

TOTAL_PRODUCTS

Step 9 – Advanced Visual (Decomposition Tree)

Insert a **Decomposition Tree**.

Analyze

TOTAL_PRODUCTS

Explain By

CATEGORY

PRICE_SEGMENT

PRICE_BAND

Users can drill into product counts by category, then by segment, then by band.

Step 10 – Dynamic Page Title

Create:

Selected Price Segment =

IF(

HASONEVALUE('GOLD_PRODUCT_RATINGS'[PRICE_SEGMENT]),

SELECTEDVALUE('GOLD_PRODUCT_RATINGS'[PRICE_SEGMENT]),

"All Price Segments"

)

Then:

Pricing Page Title =

"Pricing Analysis - " &

[Selected Price Segment]

Use it as the page title via conditional formatting.

Final Layout

Price Segment Category

Avg Price Products Avg Rating Premium Products

Products by Segment Products by Price Band

Scatter Plot Pricing Matrix

Treemap Donut Chart

Insights This Page Provides

A business user can quickly answer:

- Which price segment contains the most products?
 - Which price bands dominate the catalog?
 - Do premium products receive higher ratings?
 - Which categories have the highest average prices?
 - Are expensive products also the most reviewed?
-

Page 4 Checklist

- ☒ Price segment slicer

- ✓ Category slicer
- ✓ KPI cards
- ✓ Products by price segment chart
- ✓ Products by price band chart
- ✓ Price vs. rating scatter plot
- ✓ Pricing matrix
- ✓ Treemap
- ✓ Donut chart
- ✓ Decomposition tree
- ✓ Dynamic page title